Database Design & Implementation Answer Pack

COM3229 / COM3629 · Coronel & Morris, *Database Systems* 14e

1. **Part A**: Chapter 14 Big Data & NoSQL: all 18 review questions
2. **Part B**: 2024 Second Semester Main Examination: full memo (Q1–Q5)
3. **Part C**: 2022 Second Semester Main Examination: full memo (Q1–Q5)

> **About this memo.** These are model answers written to the textbook, not an official marking memo. Mark allocations from the papers are shown in brackets. Answers aim for roughly one point per mark. The standard SQL (2024 Q1, 2022 Q1) was run against the exam data in SQLite. The Oracle PL/SQL (Q2) was desk-checked only, because no Oracle server was available. Where a question itself has a problem (a column name that doesn't match the table, a query that returns nothing on the given data), the answer says so. Point it out in the exam too; it shows understanding.

Part A: Big Data & NoSQL Review Questions (Chapter 14)

1. What is Big Data? Give a brief definition.

Big Data refers to data sets so large, generated so quickly, or so varied in format that traditional relational DBMSs and tools cannot capture, store, manage and analyse them within acceptable time or cost. It is usually characterised by the **3 Vs**: volume, velocity and variety.

2. What are the traditional 3 Vs of Big Data? Briefly define each.

- **Volume**: the quantity of data to be stored, now measured in terabytes, petabytes and beyond. It is too much for a single server to hold or process economically.
- **Velocity**: the speed at which data is generated *and* must be processed (e.g. clickstreams, sensors, social media). It covers both how fast data arrives and how fast it must be acted on.
- **Variety**: the range of data formats: structured (tables), semi-structured (JSON, XML, logs) and unstructured (text, images, video, audio). Most Big Data does not fit a predefined relational schema.

3. Explain why companies like Google and Amazon were among the first to address the Big Data problem.

Their core business is the web, so they hit Big Data before everyone else:

- They capture enormous volumes of user-generated data (searches, clicks, purchases, reviews) arriving at very high velocity, around the clock, from millions of users.
- Their business model *depends* on analysing that data quickly (search ranking, recommendations, advertising), so they could not ignore it.
- Scaling up a single relational server was physically limited and far too expensive at their scale. They had to **scale out** over thousands of cheap commodity machines.
- No suitable product existed, so they built their own: Google's GFS and MapReduce (which inspired Hadoop) and Bigtable; Amazon's Dynamo. These became the foundations of the Hadoop and NoSQL movement.

4. Explain the difference between scaling up and scaling out.

	Scaling up (vertical)	Scaling out (horizontal)
How	Move to a bigger, more powerful single server (more CPUs, RAM, disk)	Add more servers (usually cheap commodity machines) to a cluster and spread data and work across them
Limit	Hard physical ceiling; cost rises steeply at the high end	Practically unlimited; cost grows roughly linearly
Complexity	Simple: one database, no distribution issues	Needs distributed data management, replication and fault handling
Fit	Traditional relational DBMS	Hadoop, NoSQL, Big Data

5. What is stream processing, and why is it sometimes necessary?

Stream processing analyses data *as it arrives*, before it is stored, to decide which data to keep, which to discard, and whether to act immediately. It is necessary when data arrives too fast, or in too large a volume, to store all of it first and analyse later. Examples: the CERN Large Hadron Collider discards most sensor readings in real time; fraud detection must flag a card transaction while it is happening; IoT sensor feeds.

6. How is stream processing different from feedback loop processing?

- **Stream processing** focuses on the **input**: filtering and processing data in motion to decide what is stored.
- **Feedback loop processing** focuses on the **output**: analysing (often already stored) data to produce actionable results that feed back into the business or the user's next interaction. The loop is capture → store → analyse → act. Example: an online shop analyses your browsing and purchases to recommend products while you are still shopping.

In short, stream processing decides *what data to keep*; feedback loop processing turns data into *decisions and actions*.

7. Explain why veracity, value, and visualization can also be said to apply to relational databases as well as Big Data.

These "extra Vs" are general data-management concerns, not properties that define Big Data:

- **Veracity** (trustworthiness and quality): every database must contain accurate, consistent data. Integrity constraints and normalisation exist in relational systems for exactly this reason.
- **Value**: any data, relational or not, is only worth storing if it produces useful insight or supports decisions.

- **Visualization**: results from any database must be presented in an understandable form (reports, dashboards, BI tools).

Volume, velocity and variety distinguish Big Data from traditional data. Veracity, value and visualization apply to *all* data, so they do not define Big Data.

8. What is polyglot persistence, and why is it considered a new approach?

Polyglot persistence means an organisation uses several different data-storage technologies side by side, choosing for each job the one that suits it best. For example: a relational DBMS for transactions, a key-value store for sessions and shopping carts, a document store for a product catalogue, a graph database for recommendations.

It is new because, for roughly 30 years, the relational database was the default "one size fits all" answer for almost every data need. Polyglot persistence breaks with that assumption. The trade-off is more complexity: more products to learn, integrate and keep consistent.

9. What are the key assumptions made by the Hadoop Distributed File System approach?

- **High volume**: files are very large, so data is stored in large blocks (64 MB by default, now commonly 128 MB) rather than small disk blocks. This keeps metadata small.
- **Write-once, read-many**: files are written once and then read many times, rarely updated. This removes most concurrency-control problems.
- **Streaming access**: files are read sequentially, start to finish, in batch. Throughput matters more than the latency of random access.
- **Fault tolerance**: on clusters of thousands of commodity machines, hardware failure is normal, not exceptional. Each block is therefore replicated (default 3 copies) on different nodes.

10. What is the difference between a name node and a data node in HDFS?

Name node	Data node
Stores only the **metadata**: the file namespace, which blocks make up each file, and which data nodes hold each block and its replicas	Stores the **actual data blocks** on its local disks
Normally one per cluster (master)	Many per cluster (workers)
Clients ask it *where* data is; it tells data nodes to create, delete or re-replicate blocks	Clients read and write blocks directly with data nodes; the data never flows through the name node
Detects failures through missing heartbeats and re-replicates lost blocks	Sends a **heartbeat** to the name node (every ~3 s) and a **block report** (every ~6 h)

11. Explain the basic steps in MapReduce processing.

1. **Job submission and splitting**: a client submits a job. The input data, already stored in HDFS blocks, is divided into splits, one map task per block.
2. **Map**: map tasks run in parallel, ideally on the data nodes that hold the blocks (data locality). Each map function reads its records and emits intermediate **key-value pairs**. For a word count, each word becomes `(word, 1)`.
3. **Shuffle and sort**: intermediate pairs are sorted and grouped by key, and all values for the same key are sent to the same reducer.
4. **Reduce**: each reduce function aggregates the values for a key (summing the 1s gives `(word, total)`) and writes the final result back to HDFS.
5. **Coordination**: in Hadoop 1, a *job tracker* schedules tasks onto *task trackers* on the data nodes and reruns any failed task on another node holding a replica. In Hadoop 2, YARN does this job.

12. Briefly explain how HDFS and MapReduce are complementary to each other.

- **HDFS handles storage, MapReduce handles processing**, on the same cluster of nodes.
- HDFS splits data into blocks spread across many data nodes. MapReduce sends the **computation to the data**: map tasks run on the nodes that already hold the blocks, so huge data sets do not have to be moved over the network.
- The name node's metadata tells the MapReduce scheduler where each block lives, which is what makes data locality possible.
- HDFS replication gives MapReduce fault tolerance: if a node fails, the task is rerun on another node holding a copy.
- Both assume large, write-once, sequentially read files processed in batch, so their design assumptions match.

13. What are the four basic categories of NoSQL databases?

1. **Key-value databases** (e.g. Redis, Riak, DynamoDB)
2. **Document databases** (e.g. MongoDB, CouchDB)

3. **Column-oriented / column family databases** (e.g. Cassandra, HBase)
4. **Graph databases** (e.g. Neo4j)

14. How are the value components of a key-value database and a document database different?

- **Key-value**: the value is **opaque** to the DBMS. It is just a blob of bytes that could be text, an image or anything. The DBMS does not understand its contents and can only store, fetch or delete it *by key*; the application must interpret it.
- **Document**: the value is a **document** (JSON, BSON or XML) whose internal structure the DBMS *understands*. Documents need not share a schema, but the DBMS can query, filter and index on the fields (tags) inside them, e.g. find all orders where `status = "shipped"`.

15. Briefly explain the difference between row-centric and column-centric data storage.

- **Row-centric**: all the values of one row are stored together in the same disk block. Retrieving or inserting a whole record is fast, which suits transaction processing (OLTP). Queries that read one column across millions of rows waste I/O reading every other column.
- **Column-centric**: all the values of one column are stored together. Analytical queries that scan a few columns across many rows (SUM, AVG, GROUP BY) read far less data, and similar values compress very well. Inserting or rebuilding a full row is slower, because its values are scattered across many blocks.

16. What is the difference between a column and a super column in a column family database?

- **Column**: the basic storage unit, a single **name-value pair** (e.g. `city: "Thohoyandou"`), roughly like one cell.
- **Super column**: a **named group of related columns** that are logically one unit, e.g. an `address` super column made up of the columns street, city and postal code. It is similar to a composite attribute.

A **column family** is then a collection of columns and/or super columns related to a row key.

17. Explain why graph databases tend to struggle with scaling out.

Graph databases store highly interrelated data, and their value comes from **traversing relationships** (edges) from node to node. To scale out, the graph must be split (sharded) across servers. Any split cuts edges, so a traversal that crosses a partition boundary needs a network hop, which destroys the performance that graphs exist to provide. Well-connected graphs rarely have clean cut points. Graph databases are also **not aggregate aware** (see Q18): there is no natural self-contained unit to place on one server. Many also support ACID transactions, which are harder to guarantee across a cluster.

18. Explain what it means for a database to be aggregate aware.

An **aggregate-aware** database stores together, as a single unit, all the data that the application normally reads and writes together, organised around a central entity. For example, one *order* document contains its customer details and all its line items. The data model is designed around how the data will be *used*.

- Key-value, document and column family databases are aggregate aware. Relational and graph databases are **aggregate ignorant**: data is decomposed into normalised tables and reassembled with joins.
- **Benefit**: each aggregate is self-contained, so it can be stored, retrieved and distributed (sharded) to one node as a unit. This makes scaling out easy and fast for the expected access path.
- **Cost**: it optimises for one access pattern. Queries that cut across aggregates (e.g. total sales per product across all orders) are awkward, and data is often duplicated across aggregates.

Part B: 2024 Main Examination Memo

Question 1: SQL on EMPLOYEE / DEPARTMENT / EMPLOYEE_TELEPHONE [20]

(a) Employees between 20 and 40 years old (2)

```
SELECT *
FROM   EMPLOYEE
WHERE  AGE BETWEEN 20 AND 40;
```

Result: E1 (J Smith, 21) and E2 (J Smith, 31). `BETWEEN` is inclusive.

(b) Over 21 and name starts with "Sm" (2)

```
SELECT *
FROM   EMPLOYEE
WHERE  AGE > 21
AND    NAME LIKE 'Sm%';
```

> **Trap in the data:** this returns **no rows**, because every NAME begins with an initial ("J Smith"), not "Sm". This is the expected answer to the question as written, and it is worth adding one line saying why it returns nothing. If the intent is surnames beginning with Sm: `NAME LIKE '% Sm%'`, which returns E2 (J Smith, 31). E1 is excluded because 21 is not *over* 21.

(c) Departments with a manager (2)

```
SELECT *
FROM   DEPARTMENT
WHERE  MANAGER IS NOT NULL;
```

Result: D1 and D2. Use `IS NOT NULL`; `MANAGER <> NULL` is always unknown and returns nothing.

(d) Nested SELECT: employees older than the oldest employee in D1 (3)

```
SELECT NAME
FROM   EMPLOYEE
WHERE  AGE > (SELECT MAX(AGE)
              FROM   EMPLOYEE
              WHERE  DEPT_NO = 'D1');
```

The subquery returns 31, so the result is R Brown (65) and M Green (52). Equivalent: `WHERE AGE > ALL (SELECT AGE FROM EMPLOYEE WHERE DEPT_NO = 'D1')`.

(e) Change E1's name to J Smyth (2)

```
UPDATE EMPLOYEE
SET    NAME = 'J Smyth'
WHERE  EMP_NO = 'E1';
```

Filter on the primary key, not on `NAME = 'J Smith'`, which would also rename E2.

(f) Create OLD-EMPLOYEES and insert employees over 50 (4)

```
CREATE TABLE OLD_EMPLOYEES (
    EMP_NO  VARCHAR(5)   NOT NULL,
    NAME    VARCHAR(30),
    PRIMARY KEY (EMP_NO)
);

INSERT INTO OLD_EMPLOYEES (EMP_NO, NAME)
SELECT EMP_NO, NAME
FROM   EMPLOYEE
WHERE  AGE > 50;
```

Inserts E3 (R Brown) and E5 (M Green). A hyphen is not legal in an unquoted SQL identifier, so use `OLD_EMPLOYEES` (or quote it as `"OLD-EMPLOYEES"`). One-step alternative: `CREATE TABLE OLD_EMPLOYEES AS SELECT EMP_NO, NAME FROM EMPLOYEE WHERE AGE > 50;`

(g) Delete all employees over 60 (2)

```
DELETE FROM EMPLOYEE
WHERE  AGE > 60;
```

Deletes E3 (R Brown). Note: if EMPLOYEE_TELEPHONE.EMP_NO is a foreign key to EMPLOYEE, this delete is rejected while E3 still has a telephone row. Delete the child row first, or define the FK with `ON DELETE CASCADE`.

(h) View with the employee and department numbers of managers (3)

```
CREATE VIEW MANAGER_VIEW AS
    SELECT MANAGER AS EMP_NO, DEPT_NO
    FROM   DEPARTMENT
    WHERE  MANAGER IS NOT NULL;
```

Returns (E1, D1) and (E2, D2). A join version also works: `SELECT E.EMP_NO, D.DEPT_NO FROM EMPLOYEE E JOIN DEPARTMENT D ON E.EMP_NO = D.MANAGER;`

Question 2: Sequence, triggers and procedures (Oracle PL/SQL) [20]

> **Column name mismatch in the paper:** the CUSTOMER table header says `CUS_BALANCE`, but the question text says `CUST_BALANCE`. The code below follows the table (`CUS_BALANCE`). State your assumption in the exam. The question asks for an Oracle sequence, so all of Q2 is written in Oracle PL/SQL (`:NEW` / `:OLD`, `/` to run a block).

a) Sequence CUST_NUM_SEQ starting at 2000 (2)

```
CREATE SEQUENCE CUST_NUM_SEQ
    START WITH   2000
    INCREMENT BY 1
    NOCACHE;
```

Use it with `CUST_NUM_SEQ.NEXTVAL` (next number) or `CUST_NUM_SEQ.CURRVAL` (last number this session).

b) Trigger trg_updatecustbalance: add new invoice amount to balance (5)

```
CREATE OR REPLACE TRIGGER trg_updatecustbalance
AFTER INSERT ON INVOICE
FOR EACH ROW
BEGIN
    UPDATE CUSTOMER
    SET    CUS_BALANCE = CUS_BALANCE + :NEW.INV_AMOUNT
    WHERE  CUST_NUM    = :NEW.CUST_NUM;
END;
/
```

- **AFTER INSERT**: the invoice row must exist first, and the trigger only *reads* `:NEW`.
- **FOR EACH ROW** makes it a row-level trigger, so a multi-row insert updates each customer correctly.
- It updates a *different* table (CUSTOMER), so there is no mutating-table problem.

Test: `INSERT INTO INVOICE VALUES (8005, 1001, SYSDATE, 225.40);` raises Ortega's balance from 840.92 to 1066.32.

c) Procedure prc_cust_add: add a new customer (4)

```
CREATE OR REPLACE PROCEDURE prc_cust_add (
    W_LNAME   IN CUSTOMER.CUST_LNAME%TYPE,
    W_FNAME   IN CUSTOMER.CUST_FNAME%TYPE,
    W_BALANCE IN CUSTOMER.CUS_BALANCE%TYPE DEFAULT 0
) AS
BEGIN
    INSERT INTO CUSTOMER (CUST_NUM, CUST_LNAME, CUST_FNAME, CUS_BALANCE)
    VALUES (CUST_NUM_SEQ.NEXTVAL, W_LNAME, W_FNAME, W_BALANCE);
    DBMS_OUTPUT.PUT_LINE('Customer ' || W_FNAME || ' ' || W_LNAME || ' added');
END;
/

EXEC prc_cust_add('Powers', 'Ruth', 500);   -- gets CUST_NUM 2000
```

It uses the sequence from (a) for the key, so callers never supply CUST_NUM. If your lecturer expects the number as a parameter, add `W_CUS_NUM IN NUMBER` and insert that instead.

d) Trigger trg_updatecustbalance2: reverse balance when an invoice is deleted (6)

```
CREATE OR REPLACE TRIGGER trg_updatecustbalance2
AFTER DELETE ON INVOICE
FOR EACH ROW
BEGIN
    UPDATE CUSTOMER
    SET    CUS_BALANCE = CUS_BALANCE - :OLD.INV_AMOUNT
    WHERE  CUST_NUM    = :OLD.CUST_NUM;
END;
/
```

- On DELETE there is no `:NEW` row; the deleted values are only available through `:OLD`. This is the point being examined.
- It **subtracts** the amount, because (b) added it when the credit sale was recorded.
- Row level, so deleting several invoices reverses each one.

Test: `DELETE FROM INVOICE WHERE INV_NUM = 8005;` returns Ortega's balance to 840.92.

e) Procedure prc_inv_delete(invoice number) (3)

```
CREATE OR REPLACE PROCEDURE prc_inv_delete (
    W_INV_NUM IN INVOICE.INV_NUM%TYPE
) AS
BEGIN
    DELETE FROM INVOICE
    WHERE  INV_NUM = W_INV_NUM;
    DBMS_OUTPUT.PUT_LINE('Invoice ' || W_INV_NUM || ' deleted');
END;
/

EXEC prc_inv_delete(8005);
```

Note that the procedure does **not** touch the balance: deleting the row fires `trg_updatecustbalance2` automatically. Updating the balance here as well would subtract the amount twice.

Question 3: Development lifecycle, stored procedures, triggers [20]

(a) Difference between the prototyping approach and the database systems development lifecycle (4)

Database systems development lifecycle (DSDLC)	Prototyping
A structured, largely **sequential** set of stages: planning, system definition, requirements analysis, design (conceptual, logical, physical), DBMS selection, application design, implementation, data conversion and loading, testing, operational maintenance	Building a quick **working model** of the system (or part of it) that users can try out, then refining it in repeated cycles
Requirements are gathered and fixed early, then signed off; each stage produces documented deliverables	Requirements are discovered and clarified through user feedback on the prototype
Thorough and well documented, but slow; errors in requirements surface late	Fast feedback that catches misunderstandings early, but risks poor documentation and users mistaking the prototype for the finished system
Produces the complete production system	Usually a *step within* the DSDLC (requirements and design validation), not a replacement for it. It may be thrown away or evolved

(b) Main activities of the implementation stage (6)

Implementation is the physical realisation of the database and application designs:

- **Install the DBMS** and prepare the hardware, operating system and storage. Virtualisation may be used to provide the server.
- **Create the database and its schema**: run DDL (`CREATE TABLE`, constraints, domains, indexes, views) generated from the physical design, and allocate tablespaces and storage groups.
- **Implement security**: create users, roles and privileges (`GRANT` / `REVOKE`), passwords, encryption and audit trails.
- **Implement the application programs**: write and compile code, including stored procedures, triggers and embedded SQL, plus forms and reports.
- **Load or convert data**: populate the new database from the old system, or import from files, spreadsheets or other databases. This includes cleaning and validating the data.

- **Set up backup, recovery and performance settings**, and prepare for testing (test plans, sample data) and user training.

(c) Three advantages of using stored procedures (4)

1. **Reduced network traffic and better performance**: the procedure is stored and runs on the server, so the client sends one `CALL` instead of many individual SQL statements, and the code is pre-compiled.
2. **Less code duplication and easier maintenance**: business logic (e.g. "add customer", "record sale") is written once and shared by every application. A rule change is made in one place.
3. **Better security and integrity**: users can be granted `EXECUTE` on a procedure without direct access to the underlying tables, and every application enforces the same rules in the same way. Encapsulating several statements also lets them run as a single transaction.

(d) Describe a database trigger and two advantages (6)

A **trigger** is procedural SQL code that the DBMS invokes **automatically** when a specified data-manipulation event (INSERT, UPDATE or DELETE) occurs on a particular table. It has:

- a **timing**: BEFORE or AFTER
- a **triggering event**: the DML statement
- a **level**: statement level, or row level (FOR EACH ROW)
- an **action**: the code between BEGIN and END

It runs as part of the transaction that fired it, so if the trigger fails, the triggering statement is rolled back too.

Advantages (any two, explained):

1. **Automatic enforcement of business rules and integrity**: rules that ordinary constraints cannot express (e.g. reorder when stock falls below a minimum) are enforced no matter which application changes the data. Nobody has to remember to run them.
2. **Keeping redundant or derived data consistent**: e.g. updating CUS_BALANCE whenever an invoice is added or deleted (Q2), or P_QOH whenever a sale line is inserted. This keeps redundancy *controlled*.
3. **Auditing**: automatically log who changed what and when into an audit table.

Question 4: Hadoop and NoSQL [20]

(a) How HDFS and MapReduce are complementary (4)

See Part A, Q12. Key points for 4 marks:

1. HDFS provides distributed storage and MapReduce provides distributed processing, on the same cluster.
2. MapReduce moves the computation to the data nodes holding the blocks (data locality), which avoids moving huge data sets.
3. The name node's block-location metadata is what lets the scheduler place tasks next to the data.
4. HDFS replication lets MapReduce rerun failed tasks on another replica, giving fault tolerance.

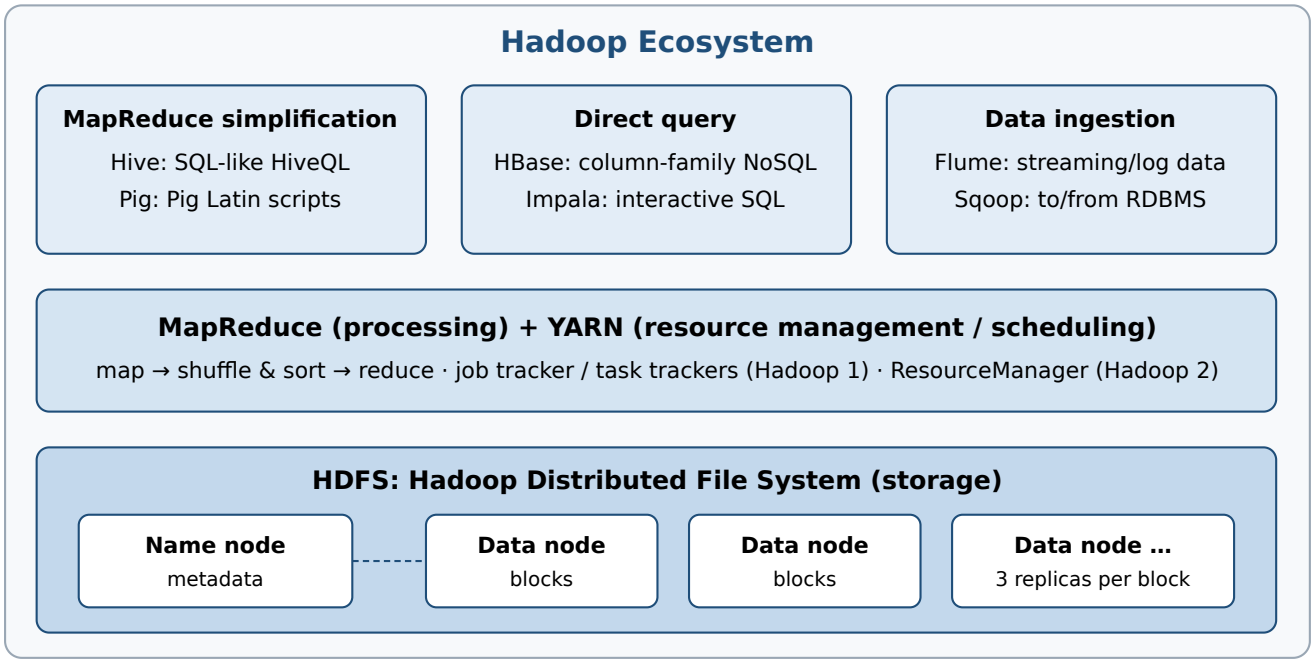
(b) Value components of key-value vs document databases (4)

See Part A, Q14. Key-value: the value is opaque to the DBMS and is accessed only by key. Document: the value is a structured, self-describing document (JSON or XML) that the DBMS can query and index by its internal fields.

(c) Name node vs data node in HDFS (4)

See Part A, Q10 (table). Name node: one master holding the metadata (namespace, block-to-node map); it coordinates replication and detects failures. Data nodes: many workers storing the actual blocks; they serve reads and writes directly to clients and send heartbeats and block reports to the name node.

(d) Key components of the Hadoop ecosystem, with a diagram (8)



Core: HDFS (storage) and MapReduce/YARN (processing). Tools layered on top: ingestion, MapReduce simplification, direct query.

- **HDFS**: distributed, replicated, block-based storage across commodity nodes (one name node, many data nodes).
- **MapReduce**: the batch processing framework that runs map and reduce tasks next to the data. **YARN** (Hadoop 2) separates cluster resource management and scheduling from MapReduce.
- **Data ingestion tools**: **Flume** collects and moves large volumes of streaming and log data into HDFS. **Sqoop** transfers bulk data between Hadoop and relational databases in both directions.
- **MapReduce simplification tools**: **Hive** is a data-warehouse layer with HiveQL, a SQL-like language compiled into MapReduce jobs. **Pig** compiles scripts in Pig Latin, a high-level procedural language, into MapReduce jobs.
- **Direct query tools**: **HBase** is a column-family NoSQL database on top of HDFS that gives fast random reads and writes, which raw HDFS cannot. **Impala** runs interactive SQL directly on HDFS data without MapReduce, giving low latency.

Question 5: Data warehousing and OLAP [20]

(a) How data marts differ from data warehouses; reasons for a data mart (4)

A **data warehouse** is an enterprise-wide, integrated, subject-oriented, time-variant, non-volatile collection of data covering the whole organisation. A **data mart** is a **small, single-subject subset** of warehouse data that supports decision making for one business unit or a small group of users (e.g. a sales mart for the marketing department).

Reasons for implementing a data mart:

- **Lower cost and shorter implementation time**: months instead of years.
- **Lower risk**: it can be a pilot or proof of concept before committing to a full warehouse, and it can grow incrementally into one.
- **Easier management of people and organisational issues**: one department's needs and politics are simpler to satisfy than the whole enterprise's.
- **Better performance and focus** for its users, because it holds only relevant data.

(b) Use of facts, dimensions and attributes in the star schema (6)

- **Facts**: numeric measurements of a business activity, such as sales units, sales amount, cost or profit. They are stored in the central **fact table**, are usually additive, and are what gets aggregated (SUM, AVG). The fact table's primary key is a **composite of foreign keys** to the dimension tables, with an M:1 relationship to each dimension.
- **Dimensions**: qualifying characteristics that give the facts context and perspective, answering by *what*, *where* and *when*: product, location, customer and time. Each dimension is a **dimension table** around the fact table, which gives the "star" shape.
- **Attributes**: the columns of the dimension tables (e.g. product brand and category; region, city and store; year, quarter and month). They are used to **search, filter, group and classify** facts, and to define **attribute**

hierarchies (year → quarter → month) that support **drill-down** and **roll-up**, and **slice and dice** analysis.

Example: "Total sales (fact) of Electronics products (product attribute) in Limpopo (location attribute) in Q3 2024 (time attribute)."

(c) OLAP client/server components and architectures (6)

See also the longer answer in Part C, Q5(c). OLAP has three main **logical components**, which can sit on the client, the server, or be split between them:

1. **Graphical user interface (GUI)**: the front end where users build queries, pivot, drill down and see charts. It usually runs on the client and is often a familiar tool such as a spreadsheet add-in or web dashboard.
2. **Analytical processing logic**: the OLAP engine that understands dimensions and hierarchies, translates user requests into queries, and performs aggregation, drill-down/roll-up and calculations.
3. **Data-processing logic**: extracts and filters data from the data warehouse, operational databases and external sources (via the DBMS), and supplies it to the analytical logic.

Architectures:

- **ROLAP**: OLAP functionality over a relational database using star schemas. Scales to very large databases and allows ad hoc queries.
- **MOLAP**: data stored in proprietary multidimensional arrays (data cubes) held in a cube cache. Faster for predefined dimensions, but less flexible.
- **Deployment**: most commonly a central **OLAP server** with a thin client GUI. A variant uses **local mini data marts** on the client for speed and offline work.

It fits the existing environment because it uses the existing client/server network and DBMSs as data sources.

(d) Defining multidimensional data analysis for users and its advantages (4)

For users: *"Multidimensional analysis lets you look at a business measure, such as sales, from several perspectives at once (by product, by region, by month), the way you already think about the business. You can view it as a cube you rotate and slice: drill down from year to month, roll up from store to region, or swap rows and columns, without writing queries."*

Advantages:

- It matches how managers naturally think, so it is intuitive.
- It is fast and interactive, with no IT request needed for each new view.
- It reveals trends, comparisons and exceptions that flat reports hide.
- It is built on consistent, integrated warehouse data, so everyone works from "one version of the truth".

Part C: 2022 Main Examination Memo

Question 1: SQL on the FACT library ERD [20]

Tables from the ERD: BOOK(Book_Num, Book_Title, Book_Year, Book_Cost, Book_Subject, Pat_ID), PATRON(Pat_ID, Pat_FName, Pat_LName, Pat_Type), AUTHOR(Au_ID, Au_FName, Au_LName, Au_BirthYear), WRITES(Book_Num, Au_ID), CHECKOUT(Check_Num, Book_Num, Pat_ID, Check_Out_Date, Check_Due_Date, Check_In_Date). BOOK.Pat_ID is the deliberately redundant "currently checked out by" foreign key.

a) Title, cost and year of every book, sorted by title (3)

```
SELECT   BOOK_TITLE, BOOK_COST, BOOK_YEAR
FROM     BOOK
ORDER BY BOOK_TITLE;
```

b) Number, title and cost of books costing $59.95, sorted by number (3)

```
SELECT   BOOK_NUM, BOOK_TITLE, BOOK_COST
FROM     BOOK
WHERE    BOOK_COST = 59.95
ORDER BY BOOK_NUM;
```

c) Middleware or Cloud books costing more than $70, sorted by number (5)

```
SELECT   BOOK_NUM, BOOK_TITLE, BOOK_SUBJECT, BOOK_COST
FROM     BOOK
WHERE    BOOK_SUBJECT IN ('Middleware', 'Cloud')
AND      BOOK_COST > 70
ORDER BY BOOK_NUM;
```

> **This is where the marks are:** if you write it with OR, you **must** use brackets: `WHERE (BOOK_SUBJECT = 'Middleware' OR BOOK_SUBJECT = 'Cloud') AND BOOK_COST > 70`. Without them, AND is evaluated first, so *every* Middleware book is returned regardless of price. This was confirmed by test: a $49.95 Middleware book wrongly appeared in the result.

d) Patrons whose last name begins with "C", sorted by patron ID (3)

```
SELECT   PAT_ID, PAT_FNAME, PAT_LNAME, PAT_TYPE
FROM     PATRON
WHERE    PAT_LNAME LIKE 'C%'
ORDER BY PAT_ID;
```

e) Author last name, first name and book number for each book written, sorted (6)

```
SELECT   A.AU_LNAME, A.AU_FNAME, W.BOOK_NUM
FROM     AUTHOR A
JOIN     WRITES W ON A.AU_ID = W.AU_ID
ORDER BY A.AU_LNAME, A.AU_FNAME, W.BOOK_NUM;
```

WRITES is the bridge (composite) entity that resolves the M:N relationship between AUTHOR and BOOK. Book_Num is already in WRITES, so the BOOK table is not needed. Older join syntax is also acceptable: `FROM AUTHOR A, WRITES W WHERE A.AU_ID = W.AU_ID`.

Question 2: Simpleco sequence, triggers and procedures [20]

> This question is **word-for-word identical** to 2024 Q2. See **Part B, Question 2** for (a) the sequence, (b) `trg_updatecustbalance`, (c) `prc_cust_add`, (d) `trg_updatecustbalance2` and (e) `prc_inv_delete`, including the CUS_/CUST_BALANCE naming note. It has now appeared in two papers, so learn it until you can write it from memory.

Question 3: DBA skills and Hadoop [20]

a) Four technical skills desired for DBA personnel (8)

1. **Broad data-processing background and up-to-date knowledge of database technologies**: understands hardware, operating systems, networks and current DBMS products and features, so the DBA can evaluate, install and tune them and judge new technology such as cloud and NoSQL.
2. **Understanding of the Systems Development Life Cycle (SDLC) and structured methodologies**: can take part in each phase of system development and use analysis and design tools (data flow diagrams, structure

charts, programming languages) to co-ordinate with analysts and programmers.
3. **Knowledge of the Database Life Cycle (DBLC)**: can guide a database through initial study, design, implementation and loading, testing and evaluation, operation, and maintenance and evolution.
4. **Database modelling and design skills**: conceptual, logical and physical design (ER modelling, normalisation, choosing indexes and storage) so that the database meets integrity, security and performance requirements.
5. **Operational skills**: hands-on database implementation, data dictionary management, security (users, roles, privileges), backup and recovery, performance monitoring and tuning, and SQL and procedural SQL.

Any four, each described, earn 2 marks apiece. Do not list *managerial* skills (communication, negotiation, planning); the question asks for technical ones.

b) Name node vs data node in HDFS (4)

See Part A, Q10. The name node is the single master holding metadata only (namespace, which blocks form each file, where each replica lives); it co-ordinates replication. Data nodes are the many workers that store the actual blocks, serve client reads and writes directly, and report to the name node with heartbeats (~3 s) and block reports (~6 h).

c) The need for a Hadoop ecosystem, and its key components (8)

Why it is needed:

- HDFS and MapReduce alone are low level. Writing MapReduce jobs in Java is slow and needs programming skill most analysts do not have.
- MapReduce is batch-only and high latency, so it cannot answer interactive queries or give fast random record access.
- Data must be moved in and out of Hadoop from many sources: logs, streams and relational databases.

An ecosystem of complementary tools grew around the core to fill these gaps, making Hadoop usable as a complete Big Data platform.

Key components: see the diagram and list in Part B, Q4(d):

- **Core**: HDFS (storage), MapReduce (processing), YARN (resource management).
- **Ingestion**: Flume, Sqoop.
- **MapReduce simplification**: Hive, Pig.
- **Direct query**: HBase, Impala.

Question 4: Magazine company: distributed database design [20]

a) Recommendations on the type and characteristics of the database system (4)

- **Use a distributed DBMS (DDBMS)**: four regional sites each own and process their own data locally, while headquarters needs a global view. A single centralised database would bring back the problems management wants to remove (bottleneck, communication cost, single point of failure).
- **Horizontally fragment** CUSTOMER and INVOICE by province/region, with each fragment stored at its own regional node so that most transactions are local.
- **Distribution transparency**, at least location and fragmentation transparency: headquarters must be able to run "all customers by region" as one query without knowing where the fragments are. The DDBMS reassembles them (UNION).
- **Transaction transparency and distributed concurrency control** (two-phase commit) for any updates spanning sites, plus **failure transparency** so that one site going down does not stop the others.
- Preferably **homogeneous** (the same DBMS at all sites) for simpler administration. **Replication** of read-mostly data to headquarters is optional if reporting load becomes heavy.

b) Horizontal fragments by PROVINCE (CUSTOMER) and REGION (INVOICE), with node names, location, fragment names, attributes and demo data (16)

Fragment allocation. Node names and cities are illustrative; any sensible naming earns the marks.

Fragment	Location	Node	Condition	Attributes
CUST_KZN	KwaZulu-Natal (Durban)	DBN	CUS_PROVINCE = 'KZN'	CUS_NUM, CUS_NAME, CUS_ADDRESS, CUS_CITY, CUS_PROVINCE, CUS_ZIP, CUS_SUBSDATE (all attributes)
CUST_MP	Mpumalanga (Mbombela)	MBO	CUS_PROVINCE = 'MP'	
CUST_LP	Limpopo (Polokwane)	PLK	CUS_PROVINCE = 'LP'	
CUST_GP	Gauteng (Johannesburg)	JHB	CUS_PROVINCE = 'GP'	
INV_KZN	KwaZulu-Natal (Durban)	DBN	INV_REGION = 'KZN'	INV_NUM, INV_REGION, CUS_NUM, INV_DATE, INV_TOTAL (all attributes)
INV_MP	Mpumalanga (Mbombela)	MBO	INV_REGION = 'MP'	
INV_LP	Limpopo (Polokwane)	PLK	INV_REGION = 'LP'	
INV_GP	Gauteng (Johannesburg)	JHB	INV_REGION = 'GP'	

Demonstration data (each fragment has the same columns but different rows):

CUST_KZN · Node DBN · Durban

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_CITY	PROV	ZIP	SUBSDATE
10010	S. Mkhize	12 Smith St	Durban	KZN	4001	2022-02-01
10011	T. Naidoo	4 Church Rd	Pietermaritzburg	KZN	3201	2022-05-01

CUST_MP · Node MBO · Mbombela

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_CITY	PROV	ZIP	SUBSDATE
20020	L. Mahlangu	8 Brown St	Mbombela	MP	1201	2022-03-01
20021	J. Nkosi	21 Main Rd	Witbank	MP	1035	2022-07-01

CUST_LP · Node PLK · Polokwane

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_CITY	PROV	ZIP	SUBSDATE
30030	R. Mudau	5 Thavhani Rd	Thohoyandou	LP	0950	2022-01-01
30031	P. Mokoena	17 Grobler St	Polokwane	LP	0699	2022-06-01

CUST_GP · Node JHB · Johannesburg

CUS_NUM	CUS_NAME	CUS_ADDRESS	CUS_CITY	PROV	ZIP	SUBSDATE
40040	K. Botha	3 Jan Smuts Ave	Johannesburg	GP	2196	2022-04-01
40041	N. Dlamini	9 Church St	Pretoria	GP	0002	2022-08-01

INV_KZN · Node DBN

INV_NUM	REGION	CUS_NUM	INV_DATE	INV_TOTAL
1001	KZN	10010	2023-02-01	360.00
1002	KZN	10011	2023-05-01	360.00

INV_LP · Node PLK

INV_NUM	REGION	CUS_NUM	INV_DATE	INV_TOTAL
3001	LP	30030	2023-01-01	320.00
3002	LP	30031	2023-06-01	320.00

INV_MP · Node MBO

INV_NUM	REGION	CUS_NUM	INV_DATE	INV_TOTAL
2001	MP	20020	2023-03-01	340.00
2002	MP	20021	2023-07-01	340.00

INV_GP · Node JHB

INV_NUM	REGION	CUS_NUM	INV_DATE	INV_TOTAL
4001	GP	40040	2023-04-01	380.00
4002	GP	40041	2023-08-01	380.00

Creating a fragment (repeat for each province or region):

```
CREATE TABLE CUST_LP AS SELECT * FROM CUSTOMER WHERE CUS_PROVINCE = 'LP';
CREATE TABLE INV_LP  AS SELECT * FROM INVOICE  WHERE INV_REGION   = 'LP';
```

Headquarters queries: with fragmentation transparency the user writes against the global CUSTOMER table; behind the scenes the DDBMS reconstructs it:

```
-- "List all current customers by region"
SELECT * FROM CUST_KZN UNION ALL SELECT * FROM CUST_MP
UNION ALL SELECT * FROM CUST_LP UNION ALL SELECT * FROM CUST_GP
ORDER BY CUS_PROVINCE;
```

Correctness of the design (worth stating for full marks):

- **Completeness**: every row belongs to exactly one of the four provinces.
- **Disjointness**: no row appears in two fragments, since the conditions are mutually exclusive.
- **Reconstruction**: the UNION of the fragments returns the original table.

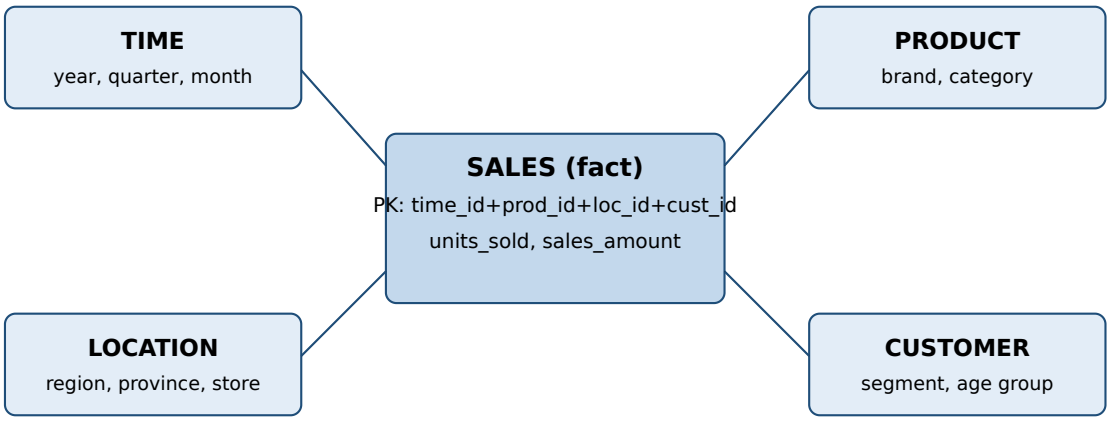
Each fragment has the same attributes but a different number of rows.

> **Point worth raising:** INV_REGION duplicates the customer's province. INVOICE should be fragmented *with* its customer (derived horizontal fragmentation), so that an invoice always lives on the same node as its customer. If a customer moves province, both rows must move, or the customer-invoice join becomes a cross-site query. A CHECK or trigger keeping INV_REGION equal to CUS_PROVINCE protects this.

Question 5: Data warehouse project [20]

a) How you would use a star schema in the design (4)

1. **Identify the facts** the business wants to analyse (e.g. units sold and sales amount) and build a central **fact table** at a chosen grain (e.g. one row per product per store per day).
2. **Identify the dimensions** users analyse those facts by (product, location, customer, time) and build one **dimension table** for each, with a surrogate key.
3. **Define attributes and hierarchies** in each dimension (year → quarter → month; region → province → store) to support filtering, drill-down and roll-up.
4. **Link them**: the fact table's primary key is the composite of the dimension foreign keys (M:1 to each dimension). Then improve performance as needed: snowflake (normalise) a dimension, add aggregate fact tables, or partition and replicate.



Star schema: one central fact table, M:1 to each surrounding dimension table.

b) Defining multidimensional data analysis for users, and its advantages (7)

Definition for users: multidimensional data analysis processes and views business data as a **cube**. A measure such as sales sits at the intersection of several **dimensions**, such as product, region and time, and can be examined from any of them. You can:

- **slice** the cube (only 2024)
- **dice** it (Limpopo, Electronics, Q3)
- **drill down** (year → month) or **roll up** (store → province)
- **pivot** (swap rows and columns)

All of this happens in point-and-click tools, without writing SQL.

Advantages to sell:

- **Matches how managers think**: business questions are naturally multidimensional ("sales by product, by region, by quarter"), so the tool is intuitive.
- **Advanced presentation**: cross-tabs, pivot tables, charts and dashboards make trends and exceptions visible at a glance.
- **Advanced aggregation and computation**: totals, subtotals, percentages, rankings, period-over-period comparisons and forecasting are built in.
- **Advanced data modelling**: what-if and scenario analysis.
- **Speed and self-service**: pre-aggregated, integrated warehouse data gives fast answers to ad hoc questions without waiting for IT to write each report.
- **Consistent information**: everyone analyses the same integrated, historical, cleaned data, which supports better and faster decisions and lets users spot trends over time.

c) OLAP overview: main client/server components and architectures (9)

Three main components. These are logical layers; they can run on the client, the server, or be split between them:

1. **Graphical user interface (GUI)**: the front end where users navigate dimensions, drill down, pivot and see charts. It is usually on the client, often a familiar tool such as a spreadsheet, BI dashboard or web browser, which keeps OLAP easy to use and consistent with the existing desktop environment.
2. **Analytical processing logic**: the OLAP "engine". It understands the dimensional model (dimensions, hierarchies, measures), interprets user requests, and performs the multidimensional work: aggregation, drill-down/roll-up, calculations and comparisons. It hides the complexity of the data source from the user.
3. **Data-processing logic**: retrieves the data. It extracts, filters and integrates data from the data warehouse, operational databases, flat files and external sources through the DBMS, and passes it to the analytical logic. It must give rapid, consistent response times on very large databases.

Client/server architectures:

- **OLAP server architecture** (most common): the analytical and data-processing logic runs on a central OLAP server attached to the data warehouse, and clients run only the GUI (a thin client). This gives centralised control, consistent results and support for large data sets.
- **OLAP server with local mini data marts**: subsets of warehouse data are downloaded to the client and analysed locally. This is fast, works offline and reduces server load, but the copies can become out of date or inconsistent, and there are more data copies to secure.
- **ROLAP (relational OLAP)**: the server stores data in a relational DBMS using star schemas, with extensions for multidimensional schema support, optimised query access and VLDB support. It is flexible (ad hoc queries, dimensions can be added dynamically), uses existing relational skills and DBMS, and scales to medium and large data sets.
- **MOLAP (multidimensional OLAP)**: the server stores data in proprietary multidimensional arrays (data cubes, or hypercubes), often held in memory in a cube cache. It is very fast for predefined dimensions, but adding a dimension means rebuilding the cube, access languages are proprietary, and sparse data wastes space.
- **HOLAP** (hybrid): detail data stays in relational tables (ROLAP) while summaries are held in cubes (MOLAP), combining the strengths of both.

How it fits the existing environment: OLAP is itself client/server. It reuses the organisation's existing network, desktop tools and relational DBMSs, and reads from the data warehouse as its main source, so it extends the current architecture rather than replacing it.